

Projekt: Krasnal

Eryk Daczyński
Małgorzata Maćkowiak
Jeremiasz Wacławski
Aleks Zieliński

April 28, 2026

Contents

1	Informacje wstępne	3
2	Funkcje	3
2.1	det()	3
2.2	cmpDouble() i sgn()	3
2.3	is_point_on_segment()	4
2.4	do_segments_intersect()	4
3	Struktury	4
3.1	Point	4
4	Testy	5
5	Prezentacja działania	5

1 Informacje wstępne

```
1 int main(){
2     int i = 0;
3
4     return i;
5 }
```

Listing 1: x example

2 Funkcje

2.1 det()

```
1 #ifndef DET_H
2 #define DET_H
3
4 #include "../Point.h"
5
6 double det(Point p1, Point p2, Point p3);
7
8 #endif
```

Listing 2: det.h

```
1 #include "det.h"
2
3 double det(Point p1, Point p2, Point p3) {
4     // 3 x 3
5     // D = (x2 - x1)(y3 - y1) - (y2 - y1)(x3 - x1)
6
7     /*
8         | 1 x1 y1 | = p1
9         | 1 x2 y2 | = p2
10        | 1 x3 y3 | = p3
11
12        D > 0 => p3 na lewo od p1p2
13        D < 0 => p3 na prawo od p1p2
14        D = 0 => p3 "na" p1p2 (wspol liniowy)
15
16        */
17
18     return (p2.x - p1.x) * (p3.y - p1.y) - (p2.y - p1.y) * (p3.x - p1.x);
19 }
```

Listing 3: det.cpp

2.2 cmpDouble() i sgn()

```
1 #ifndef EPS_H
2 #define EPS_H
3
4 constexpr double EPS = 1e-9;
5
6 inline int cmpDouble(double a, double b) {
7     if (a < b - EPS) return -1;
8     if (a > b + EPS) return 1;
9     return 0;
10 }
11
12 inline int sgn(double x) {
13     return cmpDouble(x, 0.0);
14 }
15
```

```
16 #endif
```

Listing 4: EPS.h

2.3 is_point_on_segment()

```
1 #include "is_point_on_segment.h"
2
3 // Sprawdzenie czy punkt P nalezy do odcinka p1p2
4 bool is_point_on_segment(Point p1, Point p2, Point P)
5 {
6     // P nalezy gdy p1 p2 i P sa wspolliniowe -> D = 0
7     if(sgn(det(p1, p2, P)) != 0){
8         return false;
9     }
10    // P nalezy gdy min(x1,x2) <= x3 <= max(x1,x2)
11    // oraz max(y1, y2) <= y3 <= max(y1, y2)
12    bool x_ok = cmpDouble(std::min(p1.x, p2.x), P.x) <= 0 &&
13               cmpDouble(P.x, std::max(p1.x, p2.x)) <= 0;
14    bool y_ok = cmpDouble(std::min(p1.y, p2.y), P.y) <= 0 &&
15               cmpDouble(P.y, std::max(p1.y, p2.y)) <= 0;
16
17    return x_ok && y_ok;
18 }
```

Listing 5: is_point_on_segment.cpp

2.4 do_segments_intersect()

```
1 #include "do_segments_intersect.h"
2
3 // Sprawdzenie czy odcinki p1p2 p3p4 sie przecinaja
4 bool do_segments_intersect(Point p1, Point p2, Point p3, Point p4)
5 {
6     double d1 = det(p1,p2,p3);
7     double d2 = det(p1,p2,p4);
8     double d3 = det(p3,p4,p1);
9     double d4 = det(p3,p4,p2);
10
11    // Przecinaja sie gdy:
12    // WARUNEK 2: kazdy odcinek przecina prosta wyznaczona przez drugi odcinek
13    if((sgn(d1) * sgn(d2) < 0) && (sgn(d3) * sgn(d4) < 0)) return true;
14
15    // WARUNEK 1: ktorys z koncow odc. nalezy do drugiego odc.
16    // Czy P1 lezy na P3P4 || Czy P2 lezy na P3P4 || Czy P3 lezy na P1P2 || Czy P4
17    // lezy na P1P2
18    if(is_point_on_segment(p3, p4, p1) || is_point_on_segment(p3, p4, p2) ||
19       is_point_on_segment(p1, p2, p3) || is_point_on_segment(p1, p2, p4)) {
20        return true;
21    }
22    return false;
23 }
```

Listing 6: do_segments_intersect.cpp

3 Struktury

3.1 Point

```
1 #include "Point.h"
2
3 Point::Point(double x, double y) : x(x), y(y) {}
4
5 // Operator mniejszosci
6 bool Point::operator<(const Point& other) const {
7     int cmpX = cmpDouble(x, other.x);
```

```

8     if (cmpX != 0) {
9         return cmpX < 0;
10    }
11    return cmpDouble(y, other.y) < 0;
12 }
13
14 // Operator wielkosci
15 bool Point::operator>(const Point& other) const {
16     return other < *this;
17 }
18
19 // Operator rownosci
20 bool Point::operator==(const Point& other) const {
21     return cmpDouble(x, other.x) == 0 && cmpDouble(y, other.y) == 0;
22 }
23
24 // Operator nierownosci
25 bool Point::operator!=(const Point& other) const {
26     return !(*this == other);
27 }
28
29 // Operator mniejszosci lub rownosci
30 bool Point::operator<=(const Point& other) const {
31     return (*this < other) || (*this == other);
32 }
33
34 // Operator wielkosci lub rownosci
35 bool Point::operator>=(const Point& other) const {
36     return (*this > other) || (*this == other);
37 }
38
39 // Operator przypisania
40 Point& Point::operator=(const Point& other) {
41     if(this != &other){
42         x = other.x;
43         y = other.y;
44     }
45
46     return *this;
47 }
48
49 // Wypisanie punktu
50 std::ostream& operator<<(std::ostream& stream, const Point& p){
51     stream << "(" << p.x << ", " << p.y << ")";
52     return stream;
53 }

```

Listing 7: Point.cpp

4 Testy

5 Prezentacja działania